

TRẦN ĐÌNH QUẾ

GIÁO TRÌNH

PHÂN TÍCH VÀ THIẾT KẾ
HỆ THỐNG THÔNG TIN

PTIT

MỤC LỤC

CHƯƠNG 1: CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG	1
1.1 GIỚI THIỆU	1
1.2 CÁC KIỂU HỆ THỐNG THÔNG TIN	1
1.3 CÁC KHÁI NIỆM CƠ BẢN CỦA HỆ HƯỚNG ĐỐI TƯỢNG	2
1.3.1 Lớp và đối tượng	3
1.3.2 Phương thức và thông điệp	4
1.3.3 Đóng gói và ấn dấu thông tin	5
1.3.4 Đa xạ và ràng buộc động	8
1.3.5 Quan hệ giữa các lớp	8
1.4 SỬ DỤNG LẠI	17
1.5 KẾT LUẬN	19
BÀI TẬP	19
CHƯƠNG 2: MÔ HÌNH HÓA HỆ PHẦN MỀM HƯỚNG ĐỐI TƯỢNG	20
2.1 GIỚI THIỆU VỀ UML	20
2.1.1 Lịch sử phát triển của UML	20
2.1.2 UML – Ngôn ngữ mô hình hóa hướng đối tượng	20
2.1.3 Các khái niệm cơ bản trong UML	21
2.2 CÁC BIỂU ĐỒ TRONG UML	23
2.2.1 Biểu đồ ca sử dụng	24
2.2.2 Biểu đồ lớp	26
2.2.3 Biểu đồ trạng thái	32
2.2.4 Biểu đồ tuần tự	34
2.2.5 Biểu đồ giao tiếp	36
2.2.6 Biểu đồ hoạt động	38
2.2.7 Biểu đồ thành phần	40
2.2.8 Biểu đồ triển khai	41
2.3 PHƯƠNG PHÁP LUẬN PHÁT TRIỂN PHẦN MỀM	42
2.3.1 Khái niệm phương pháp luận	42
2.3.2 Các pha phát triển truyền thống	44
2.3.3 Phương pháp luận hướng đối tượng	45
2.3.4 UP	47
2.3.5 Một tiến trình phát triển phần mềm đơn giản	53
2.4 GIỚI THIỆU CÔNG CỤ PHÁT TRIỂN PHẦN MỀM	54
2.5 KẾT LUẬN	57
BÀI TẬP	57
CHƯƠNG 3: XÁC ĐỊNH YÊU CẦU	59
3.1 GIỚI THIỆU	59
3.2 CÁC BƯỚC TRONG PHA XÁC ĐỊNH YÊU CẦU	59

MỤC LỤC

3.2.1 Yêu cầu là gì?	59
3.2.2 Xác định yêu cầu	61
3.3 XÁC ĐỊNH YÊU CẦU NGHIỆP VỤ.....	62
3.3.1 Xác định và mô tả các tác nhân	63
3.3.2 Xây dựng Bảng Thuật ngữ	64
3.3.3 Xác định và mô tả các ca sử dụng nghiệp vụ	66
3.3.4 Mô tả chi tiết ca sử dụng	67
3.3.5 Xây dựng biểu đồ giao tiếp (Communication diagram)	67
3.3.6 Xây dựng biểu đồ hoạt động (Activity diagram)	68
3.4 XÁC ĐỊNH YÊU CẦU HỆ THỐNG	69
3.4.1 Xác định và mô tả các tác nhân	70
3.4.2 Xác định và mô tả các ca sử dụng	71
3.4.3 Xây dựng biểu đồ ca sử dụng	72
3.4.4 Xây dựng kịch bản	74
3.4.5 Xếp ưu tiên các ca sử dụng.....	77
3.4.6 Phác họa giao diện người dùng	78
BÀI TẬP	81
CHƯƠNG 4: PHÂN TÍCH YÊU CẦU.....	82
4.1 GIỚI THIỆU	82
4.2 CÁC BƯỚC TRONG PHA PHÂN TÍCH	83
4.3 PHÂN TÍCH TĨNH.....	84
4.3.1. Xác định các lớp.....	84
4.3.2 Xác định quan hệ giữa các lớp	85
4.3.3 Xây dựng biểu đồ lớp	85
4.3.4 Xác định thuộc tính	89
4.4 PHÂN TÍCH ĐỘNG.....	93
4.4.1 Xây dựng biểu đồ giao tiếp	94
4.4.2 Gán phương thức cho các lớp.....	96
4.5 CASE STUDY: HỆ QUẢN LÝ ĐĂNG KÝ HỌC THEO TÍN CHỈ	100
4.5.1 Xác định yêu cầu	100
4.5.2 Phân tích tĩnh.....	113
4.5.3 Phân tích động.....	119
4.6 KẾT LUẬN.....	121
BÀI TẬP	122
CHƯƠNG 5: THIẾT KẾ KIẾN TRÚC HỆ THỐNG	123
5.1 GIỚI THIỆU	123
5.2 CÁC BƯỚC TRONG PHA THIẾT KẾ	124
5.3 LỰA CHỌN CÔNG NGHỆ MẠNG CHO HỆ THỐNG	125
5.3.1 Kiến trúc mạng đơn tầng và hai tầng.....	125
5.3.2 Kiến trúc ba tầng	128
5.3.3 Kiến trúc Client – Server và kiến trúc phân tán	129
5.3.4 Biểu diễn hình trạng mạng với UML	131
5.4 THIẾT KẾ TƯƠNG TRanh VÀ AN TOÀN-BẢO MẬT	132
5.4.1 Thiết kế tương tranh	132
5.4.2 Thiết kế an toàn-bảo mật.....	133

5.5 PHÂN RÃ HỆ THỐNG THÀNH CÁC HỆ THỐNG CON.....	135
5.5.1 Hệ thống và hệ thống con.....	135
5.5.2 Các cụm (Layer).....	135
5.5.3 Ví dụ : Java Layers - Applet plus RMI	138
5.6 XÂY DỰNG BIỂU ĐỒ GÓI.....	139
5.7 KẾT LUẬN.....	140
BÀI TẬP	140
CHƯƠNG 6: THIẾT KẾ CÁC HỆ THỐNG CON	141
6.1 GIỚI THIỆU	141
6.2 XÂY DỰNG MÔ HÌNH LỚP THIẾT KẾ	141
6.2.1 Ánh xạ các phương thức.....	142
6.2.2 Các kiểu biên	142
6.2.3 Phạm vi của các trường	143
6.2.4 Các toán tử truy nhập	143
6.2.5 Ánh xạ các lớp, thuộc tính và kiểu quan hệ hợp thành	143
6.2.6 Ánh xạ các kiểu quan hệ khác.....	144
6.3 XÂY DỰNG LƯỢC ĐỒ CƠ SỞ DỮ LIỆU.....	148
6.3.1 Các hệ quản trị cơ sở dữ liệu.....	148
6.3.2 Mô hình quan hệ.....	149
6.3.3 Ánh xạ các lớp thực thể.....	150
6.3.4 Ánh xạ các liên kết.....	150
6.3.5 Ánh xạ trạng thái đối tượng.....	152
6.4 THIẾT KẾ GIAO DIỆN NGƯỜI SỬ DỤNG	155
6.5 SỬ DỤNG FRAMEWORK, MÀN HÌNH VÀ THƯ VIỆN.....	160
6.6 KẾT LUẬN.....	160
BÀI TẬP	160
PHỤ LỤC A: LỰA CHỌN CÔNG NGHỆ	161
A.1 GIỚI THIỆU	161
A.2 CÔNG NGHỆ TẦNG CLIENT.....	161
A.3 GIAO THỨC GIỮA TẦNG CLIENT VÀ TẦNG GIỮA	162
A.4 CÔNG NGHỆ TẦNG GIỮA	163
A.5 CÔNG NGHỆ TẦNG GIỮA ĐẾN TẦNG DỮ LIỆU	164
A.6 CÁC CÔNG NGHỆ KHÁC	165
PHỤ LỤC B: CASE STUDY: HỆ QUẢN LÝ ĐĂNG KÝ HỌC THEO TÍN	
CHỈ	169
B.1 XÁC ĐỊNH YÊU CẦU	169
B.2 PHÂN TÍCH TÍNH.....	201
B.3. PHÂN TÍCH ĐỘNG	207
TÀI LIỆU THAM KHẢO	227

LỜI NÓI ĐẦU

Phân tích và thiết kế các hệ thống thông tin là một môn học bắt buộc thuộc chương trình Đại học dành cho sinh viên ngành Công nghệ thông tin. Có nhiều cách tiếp cận phát triển hệ thống tùy theo kiểu ta muốn xây dựng, yêu cầu người dùng và công nghệ mà chúng ta sử dụng. Tuy nhiên, dù theo cách tiếp cận phát triển nào, các dự án phát triển hệ thống thông tin cũng phải qua các pha truyền thống sau đây: Xác định yêu cầu (requirement determination), phân tích yêu cầu (requirement analysis), thiết kế (design), cài đặt (implementation), kiểm thử (testing) và bảo trì (maintenance).

Ngày nay, cách tiếp cận hướng đối tượng càng ngày càng trở thành phổ biến trong công nghiệp phát triển phần mềm do tính hiệu quả về mặt phát triển cũng như sự hỗ trợ mạnh mẽ của nhiều công nghệ. Cách tiếp cận này xem hệ thống như một tập các lớp với các thuộc tính và thao tác hay hành vi tương ứng cùng với các tương tác giữa các đối tượng trong các lớp. Hơn nữa, sự phát triển mạnh mẽ về kỹ thuật, công nghệ, công cụ hỗ trợ và đặc biệt ngôn ngữ mô hình hóa UML (Unified Modeling Language) đã làm thay đổi căn bản quan niệm và cách phát triển hệ phần mềm. Giáo trình này được xây dựng theo chương trình đào tạo theo tín chỉ Ngành Công nghệ Thông tin tại Học viện Công nghệ Bưu chính Viễn thông. Nội dung tập trung trình bày một số vấn đề cơ bản của phân tích và thiết kế theo hướng đối tượng bao gồm từng các pha xác định yêu cầu, phân tích yêu cầu và thiết kế. Giáo trình được biên soạn dựa vào những tài liệu có sẵn được liệt kê trong phần tài liệu tham khảo và kinh nghiệm giảng dạy nhiều năm của tác giả. Ngoài những ví dụ minh họa riêng rẽ, case study Hệ quản lý học tập theo tín chỉ được sử dụng xuyên suốt trong nhiều chương nhằm giúp cho bạn đọc dễ dàng theo dõi các bước của các pha phát triển.

Mục đích của tài liệu này là nhằm phục vụ sinh viên ngành công nghệ thông tin khi học môn Phân tích và Thiết kế Hệ thống Thông tin. Tài liệu cũng có thể dành cho giảng viên tham khảo khi giảng dạy các môn học liên quan và sinh viên các ngành học khác như Điện tử - Viễn thông có thể tham khảo hay tự học để thiết kế các hệ thống thông tin thông dụng. Nội dung tài liệu bao gồm:

Chương 1: Cơ sở của phát triển phần mềm hướng đối tượng

Giới thiệu các kiểu hệ thống thông tin và mô hình hệ thống dựa vào cách tiếp cận hướng đối tượng. Các khái niệm đối tượng và lớp, đóng gói, quan hệ giữa các lớp và vấn đề sử dụng lại mã nguồn sẽ được xem xét ở mức độ vừa phải đủ để bạn đọc có cái nhìn tổng quan về những kiến thức lập trình hướng đối tượng phục vụ cho việc tìm hiểu các chương sau.

Chương 2. Mô hình hóa hệ phần mềm hướng đối tượng

Nội dung bao gồm giới thiệu về UML, các biểu đồ UML và sử dụng các biểu đồ UML trong phân tích và thiết kế hướng đối tượng. Một số phương pháp luận phát triển phần mềm hướng đối tượng hiện nay cũng sẽ được điểm qua và đặc biệt phương pháp luận UP sẽ được khảo sát chi tiết hơn. Cuối chương sẽ trình bày các pha và các bước thực hiện trong mỗi pha để phục vụ cho các chương sau này.

Chương 3. Xác định yêu cầu

Nội dung tập trung trình bày pha xác định yêu cầu dựa trên quan điểm nghiệp vụ và quan điểm người phát triển. Một case study về Quản lý đăng ký học theo tín chỉ sẽ được xem xét.

Chương 4. Phân tích yêu cầu

Nội dung bao gồm tổng quan quá trình phân tích và phân tích tĩnh cũng như phân tích động. Phần phân tích tĩnh đề cập đến việc xác định các lớp và quan hệ giữa các lớp, các thuộc tính. Phần phân tích động bàn về việc thực thi các lớp, các lớp biên, điều khiển và thực thể, các biểu đồ giao tiếp, phương thức trong lớp và cách xây dựng dựa trên gán trách nhiệm cho lớp và biểu đồ trạng thái.

Chương 5. Thiết kế kiến trúc hệ thống

Trình bày các bước trong thiết kế hệ thống, chọn topo hệ thống mạng cho thiết kế, một số chủ đề về công nghệ, thiết kế đồng thời và an toàn hệ thống. Nội dung bao gồm: Các công nghệ tầng client; Các công nghệ tầng trung gian; Các công nghệ tầng trung gian đến tầng dữ liệu; Các kiểu cấu hình; Các gói theo UML.

Chương 6. Thiết kế chi tiết

Chương này trình bày ánh xạ mô hình lớp phân tích thành mô hình lớp thiết kế, xử lý lưu trữ với cơ sở dữ liệu quan hệ, một số vấn đề liên quan đến giao diện người sử dụng, thiết kế các dịch vụ nghiệp vụ, sử dụng pattern, framework và thư viện.

Tác giả vô cùng biết ơn các đồng nghiệp thuộc Khoa Công nghệ Thông tin, Học viện Công nghệ Bru chính Viễn thông đã tạo động lực để tác giả hoàn thành tài liệu này. Trong quá trình soạn thảo giáo trình, sinh viên Khoa Công nghệ Thông tin qua nhiều thế hệ đã có nhiều đóng góp, đặc biệt góp phần xây dựng case study Hệ Quản lý Học tập theo tín chỉ. Vì vậy, tác giả muốn dành món quà này cho các bạn sinh viên trong gần 15 năm đã nuôi dưỡng niềm say mê giảng dạy và là cội nguồn của mọi cội nguồn để tác giả viết giáo trình này.

Mặc dù tác giả đã có nhiều nỗ lực để giáo trình này ra đời nhưng chắc chắn không thể tránh khỏi những thiếu sót. Tác giả rất mong nhận được nhiều ý kiến đóng góp của các đồng nghiệp cùng các bạn sinh viên để tài liệu ngày được hoàn thiện hơn.

Tác giả

PTIT

CHƯƠNG 1

CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

1.1 GIỚI THIỆU

Ngày nay, cách tiếp cận này hướng đối tượng đã được sử dụng rộng rãi cho phát triển các hệ thống phần mềm như trong quản lý doanh nghiệp, thương mại điện tử, các hệ thời gian thực, các hệ thống thông minh... Mục đích của chương này nhằm trình bày các kiểu hệ thống thông tin, những khái niệm cơ bản trong lập trình hướng đối tượng như đối tượng, lớp, đóng gói, kế thừa, đa xạ. Phần còn lại xem xét, phân tích các quan hệ giữa các lớp, các cách sử dụng lại mã nguồn hiện nay.

1.2 CÁC KIỂU HỆ THỐNG THÔNG TIN

Trong thực tế có rất nhiều kiểu hệ thống thông tin và để dễ phân loại người ta thường chia các hệ thống thông tin thành bốn mức:

- Các hệ thống điều khiển
- Các hệ cơ sở tri thức
- Các hệ thống quản lý
- Các hệ thống chiến lược

Các hệ thống điều khiển: Dành cho các nhà quản lý vận hành hay quản lý hệ thống để thu được những câu trả lời cho các câu hỏi thường ngày. Ví dụ, hệ thống lưu vết trình tự các hoạt động và giao dịch hàng ngày như các hệ xử lý giao dịch (Transaction Processing Systems), điều khiển thiết bị hay dây chuyền sản xuất, lưu trữ giao dịch, lập lịch, xử lý đơn đặt hàng...

Các hệ cơ sở tri thức: Dành cho các nhân viên tri thức và xây dựng dữ liệu nhằm giúp tổ chức, khám phá và tích hợp tri thức mới từ tri thức hiện thời vào nghiệp vụ của họ hay điều khiển luồng công việc. Ví dụ, các hệ thống hoạt động dựa trên tri thức, tự động hóa văn phòng, hệ xử lý ngôn ngữ, hệ thống tư vấn trong thương mại điện tử...

Các hệ thống quản lý: Dành cho các nhà quản lý trung gian để phục vụ việc giám sát, điều khiển, ra quyết định và các hoạt động quản trị hay hỗ trợ ra các quyết định quan trọng (ít có cấu trúc) với các yêu cầu về thông tin không rõ ràng. Ví dụ hệ thông tin quản lý, hệ hỗ trợ quyết định, hệ quản lý bán hàng cần biết hàng tồn kho, ngân sách hàng năm để lập kế hoạch sản xuất, phân tích chi phí, phân tích giá cả/lợi nhuận...

Các hệ thống chiến lược: Dành cho các nhà quản lý chính để giúp giải quyết và vạch ra các chiến lược và các xu hướng lâu dài; so sánh khả năng của tổ chức với những thay đổi của môi trường bên ngoài và cơ hội xảy ra trong khoảng thời gian dài. Ví dụ, hệ hỗ trợ

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

thực thi cho dự đoán ngân sách, xu hướng bán hàng trong năm năm, kế hoạch hoạt động trong năm năm, kế hoạch về lợi nhuận, nhân lực.

Như vậy, dựa trên bốn mức trên có thể phân thành sáu kiểu hệ thống thông tin tương ứng như sau:

- Các hệ xử lý giao dịch (Transaction Processing Systems).
- Các hệ cơ sở tri thức (Knowledge based Systems).
- Các hệ tự động hóa văn phòng (Office Automation Systems).
- Các hệ thông tin quản lý (Management Information Systems).
- Các hệ trợ giúp quyết định (Decision Support Systems).
- Các hệ trợ giúp thực thi (Executive Support Systems).

Tương ứng với các kiểu hệ thống thông tin, các quá trình xử lý và lưu trữ cũng sẽ được thể hiện khác nhau. Vì vậy, các nhà phát triển cần chú ý nắm vững những đặc trưng của kiểu hệ thống mà mình cần phải phát triển. Hơn nữa, đa phần các hệ thống thông tin trên môi trường Internet ngày nay càng trở nên phức tạp hơn, nó cần phải tích hợp nhiều kiểu hệ thống thông tin.

Ví dụ, các hệ thương mại điện tử như eBay, Amazon là những hệ phân tán không chỉ có chức năng quản lý giao dịch mua, bán hàng, bán hàng, mà còn có chức năng quản lý giao dịch thanh toán và tư vấn khách hàng... Để có thể thực hiện các chức năng này, các nhà phát triển hệ thống cần phải thiết sử dụng các công nghệ, kỹ thuật lưu trữ và xử lý phân tán cũng như nắm vững các kỹ thuật trong biểu diễn và xử lý tri thức của các lĩnh vực liên quan như trí tuệ nhân tạo, khai phá dữ liệu, web ngữ nghĩa...

1.3 CÁC KHÁI NIỆM CƠ BẢN CỦA HỆ HƯỚNG ĐỐI TƯỢNG

Phần này nhằm trình bày một cách không hình thức một số khái niệm cơ bản của các hệ hướng đối tượng như lớp, đối tượng, phương thức, thông điệp, đóng gói, ẩn dấu thông tin, kế thừa, đa xạ, ràng buộc động. Những khái niệm này giúp cho các nhà phân tích phân rã những hệ thống phức tạp thành những môđun nhỏ hơn nhằm dễ dàng quản lý, thực thi và đồng thời dễ dàng tích hợp thành một hệ thống thông tin. Tính môđun hóa này sẽ giúp cho việc phát triển thuận lợi hơn vì qua đó dễ chia sẻ với các thành viên trong nhóm và dễ dàng trao đổi với người sử dụng trong quá trình khảo sát yêu cầu. Việc môđun hóa cũng giúp nhóm phát triển xây dựng được những gói phần mềm có thể sử dụng lại cho các dự án sau này. Hơn nữa, nhiều nghiên cứu đã chỉ ra rằng cách “tư duy đối tượng” được xem là hiện thực hơn cách tư duy thuật toán hay dữ liệu vì nó thể hiện cách suy nghĩ thông thường con người về thế giới xung quanh.

1.3.1 Lớp và đối tượng

Lớp (class) là một thuật ngữ chung để xác định và tập hợp các thể hiện hay các đối tượng đặc biệt nào đó. Lớp đóng gói các đặc điểm chung của một nhóm các đối tượng. Khái niệm lớp đã được các nhà phát triển phần mềm hướng đối tượng sử dụng để mô tả các đặc trưng mà các dạng đối tượng cụ thể có thể có.

Đối tượng (object) là một khởi tạo của lớp, nó có thể là một sự vật, một thực thể, một danh từ hoặc bất cứ cái gì mà bạn có thể nhặt lên hoặc ném đi, hay những gì mà bạn có thể tưởng tượng ra với một số đặc tính nào đó của nó. Mỗi đối tượng có các *thuộc tính* (attribute) mô tả thông tin về đối tượng đó. *Trạng thái* (state) của một đối tượng được xác định bởi bộ giá trị của những thuộc tính và quan hệ với những đối tượng khác tại một thời điểm cụ thể. Mỗi đối tượng có một số *hành vi* (behavior) nhằm đặc tả những gì đối tượng này có thể thực hiện được. Trong các mô hình hướng đối tượng, các thuật ngữ hành vi, *hành động* (action), *thao tác* (operation), *phương thức* (method) đều có nghĩa như nhau nhưng thường được dùng cho các ngữ cảnh khác nhau. Ví dụ, khi nói về các đối tượng thực tế thì ta thường hay nói hành vi hay hành động của đối tượng đó; khi đề cập đến đối tượng trong lập trình người ta thường dùng phương thức hay thao tác nhưng phương thức được dùng phổ biến nó được xem là thể hiện cài đặt của hành vi.

Ví dụ, lớp Sinh viên *Sinhvien* trong Hệ quản lý học tín chỉ có các thuộc tính là mã sinh viên, họ và tên, địa chỉ... Các đối tượng như sinh viên Nguyễn Minh Ngọc có mã sinh viên NH12345, địa chỉ 17 Nguyễn Trãi, Hà nội... và có thể có các hành vi như đăng ký học, hủy đăng ký, xem lịch học... Lớp Sách *Sach* trong Hệ quản lý thư viện có các thuộc tính là tên sách *sachTen*, tên tác giả *sachTacgia*, nhà xuất bản *sachNhaXuatban*, năm xuất bản *sachNamXuatban*... Các đối tượng như cuốn sách “Phân tích và thiết kế hướng đối tượng” của tác giả Đặng Văn Đức, nhà xuất bản Giáo dục, năm 2002 có thể có các hành vi như xóa sách *xoaSach*, thêm sách *themSach*, cập nhật thông tin sách *capnhatThongtinSach*...

Biểu diễn đối tượng và lớp

Để có thể mô tả và suy nghĩ về lớp - đối tượng, chúng ta phải có cách biểu diễn chúng theo biểu đồ. Các ký hiệu biểu đồ mà chúng ta sử dụng ở đây là Biểu đồ lớp và đối tượng trong ngôn ngữ mô hình hóa UML (Chi tiết sẽ được trình bày trong Chương 2). Trong lớp Sách *Sach* được mô tả trong UML bao gồm ba thành phần là tên lớp, các thuộc tính và các phương thức kèm theo dấu ngoặc đơn như Hình 1.1:

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

Sach
sachTen sachTacgia sachNhaXuatban sachNamXuatban
xoaSach() themSach() capnhatThongtinSach()

Hình 1.1: Biểu diễn lớp sách

và đối tượng sách được biểu diễn bởi Hình 1.2

<u>Sach1: Sach</u>
sachTen = PhantichvaThietkeHuonggdoinuong sachTacgia = DangVanDuc sachNhaXuatban = Giaiduc sachNam = 2000
xoaSach() themSach() capnhatThongtinSach()

Hình 1.2: Biểu diễn đối tượng sách

Mỗi đối tượng được mô tả bởi ba thành phần tương ứng với lớp của nó:

- Tên của đối tượng có gạch chân và dấu $a:A$ thể hiện a thuộc lớp A
- Các giá trị của thuộc tính
- Các phương thức để trong dấu ngoặc thể hiện thao tác hay hành vi của đối tượng

1.3.2 Phương thức và thông điệp

Phương thức (method) cài đặt hành vi (hay còn gọi hành động, thao tác) của đối tượng. Phương thức chính là hành động mà một đối tượng có thể thực hiện và nó tương tự như một hàm hay thủ tục trong ngôn ngữ truyền thống như C, Pascal. Tuy nhiên, sự khác biệt có thể dễ thấy là phương thức phải cài đặt trong một lớp nào đó, còn hàm có thể viết bất kỳ ở đâu. *Thông điệp* (message) là thông tin được gửi cho đối tượng để kích hoạt các phương thức. Một thông điệp chính là lời gọi hàm hay thủ tục truyền thống nhưng lại từ một đối tượng này đến đối tượng khác.

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

Ví dụ, trong Hệ quản lý thư viện, nếu nhân viên thư viện cần chèn thêm một cuốn sách mới, những thông tin được đưa vào qua giao diện sẽ được hệ thống gửi đi dưới dạng thông điệp chèn cuốn sách mới cho chương trình ứng dụng. Khi đó đối tượng sách sẽ nhận thông điệp và thực hiện các công việc cần thiết gọi là thực hiện phương thức để chèn cuốn sách mới vào hệ thống.

1.3.3 Đóng gói và ẩn dấu thông tin

Các ý tưởng *đóng gói* (encapsulation) và *ẩn dấu thông tin* (information hiding) có liên quan mật thiết nhau trong các hệ hướng đối tượng. Trong khi các cách tiếp cận phát triển hệ thống thông tin truyền thống chỉ chú trọng hoặc tiến trình hoặc dữ liệu, cách tiếp cận hướng đối tượng thể hiện tính đóng gói bằng cách kết hợp cả hai tiến trình và dữ liệu vào trong một thực thể gọi là đối tượng.

Ẩn dấu thông tin thực ra đã được thể hiện trong phương pháp phát triển các hệ phần mềm theo hướng cấu trúc. Nguyên lý của ẩn dấu thông tin cho rằng chỉ thông tin được đòi hỏi để sử dụng môđun phần mềm là được công khai của sử dụng môđun đó. Nghĩa là, chỉ thông tin được yêu cầu chuyển đến môđun này và thông tin trả về từ môđun đó là được công khai mà thôi. Chúng ta thấy sự không quan tâm đến cách thức đối tượng thực hiện chức năng của nó thế nào.

Trong các hệ hướng đối tượng, việc kết hợp đóng gói thông tin và nguyên lý ẩn dấu thông tin có nghĩa rằng nguyên lý này được áp dụng vào các đối tượng thay vì chỉ áp dụng vào hàm hay tiến trình. Khi đó, các đối tượng được xem như là các hộp đen.

Trong ví dụ Hệ quản lý thư viện 1.3.2, chúng ta chỉ quan tâm đến thông điệp chèn một cuốn sách mới nhưng thuật toán bên trong cần đáp ứng với thông điệp là ẩn dấu với các thành phần khác của hệ thống. Thông tin duy nhất mà đối tượng sách này cần biết là tập các phép toán hay phương thức mà các đối tượng khác có thể tiến hành và các thông điệp nào cần phải gửi để kích hoạt chúng. Các nghiên cứu trong cách tiếp cận hướng đối tượng đã chỉ ra rằng đóng gói có một số lợi ích sau đây:

- Đóng gói là an toàn vì một đối tượng không thể can thiệp để thay đổi trạng thái tức là các giá trị thuộc tính của các đối tượng khác.
- Đóng gói làm đơn giản hóa việc chuyển đổi một lớp đang tồn tại thành một lớp được dùng để tạo các đối tượng phân tán (từ xa). Đối tượng phân tán thường nằm ở máy chủ, các phương thức của nó có thể được gọi bởi các ứng dụng nằm trên các máy khác (nói cách khác là có thể được gọi thông qua mạng). Do đó, các

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

thuộc tính thuộc đối tượng đó không thể được gọi trực tiếp như với đối tượng cục bộ. Tuy nhiên, nếu đưa vào các phương thức *set* và *get*, ta có thể làm được điều này dễ dàng.

- Ngoài ra, việc sử dụng các phương thức *set* và *get* sẽ cô lập chúng ta khỏi những thay đổi khi cài đặt chi tiết thêm một tính chất. Ví dụ, có thể đổi thuộc tính từ kiểu *int* sang kiểu *String* mà không ảnh hưởng tới các lớp khác, miễn là chúng ta thực hiện việc chuyển đổi thích hợp trong các phương thức *set* và *get*.

Ví dụ: Lớp *Employee* chứa các thông tin mà ứng dụng cần để miêu tả một nhân viên:

```
public class Employee{
    public int employeeID;
    public String firstName;
    public String lastName;
}
```

Các thuộc tính đều ghi là *public* nghĩa là chúng có thể được truy cập từ mọi lớp bằng các câu lệnh sau:

```
Employee emp = new Employee ();
emp.employeeID = 123456;
emp.firstName = "John";
emp.lastName = "Smith";
```

Mặc dù Java cho phép đọc và sửa đổi các trường hay thuộc tính theo cách này, nhưng thông thường ta không nên làm như vậy. Thay vào đó, ta cần thay đổi phạm vi truy cập tới các trường để giới hạn khả năng truy cập của chúng bằng cách sử dụng các phương thức *set*, *get* cho mỗi trường để có thể truy cập tới thuộc tính.

```
public class Employee {
    protected int employeeID;
    protected String firstName;
    protected String lastName;
    public int getEmployeeID() {
        return employeeID;
    }
    public void setEmployeeID(int id) {
        employeeID = id;
    }
    public String getFirstName() {
        return firstName;
    }
}
```

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

```
public void setFirstName(String name) {
    firstName = name;
}
public String getLastName() {
    return lastName;
}
public void setLastName(String name) {
    lastName = name;
}
}
```

Ví dụ tiếp tục sau đây chỉ ra ý nghĩa của đóng gói khi ẩn giấu việc cài đặt chi tiết

```
public class Employee {
    protected String employeeID;
    protected String firstName;
    protected String lastName;
    public int getEmployeeID() {
        return Integer.parseInt(employeeID);
    }
    public void setEmployeeID(int id) {
        employeeID = Integer.toString(id);
    }
    public String getFirstName() {
        return firstName;
    }
    public void setFirstName(String name) {
        firstName = name;
    }
    public String getLastName() {
        return lastName;
    }
    public void setLastName(String name) {
        lastName = name;
    }
}
```

Mặc dù, việc cài đặt thuộc tính `employeeID` bị thay đổi, nhưng các lớp khác khi đọc và sửa đổi thuộc tính này sẽ không nhìn thấy bất kỳ thay đổi nào trong hành vi của nó, vì việc thay đổi trong cài đặt được che giấu bởi các phương thức `set` và `get`. Đóng gói các

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

thuộc tính của lớp cho phép định nghĩa các giá trị dẫn xuất có khả năng truy cập. Ví dụ, bạn có thể định nghĩa phương thức `getFullName()` trong lớp `Employee` để trả về họ tên đầy đủ dưới dạng một chuỗi đơn:

```
public String getFullName() {  
    return firstName + " " + lastName;  
}
```

Tất nhiên, có thể lấy giá trị dẫn xuất mà không cần tạo phương thức `get`, nhưng như vậy thường sẽ tạo ra các đoạn mã trùng nhau khi sử dụng. Ví dụ, để dẫn xuất tên đầy đủ ở một số vị trí trong ứng dụng, bạn phải copy đoạn mã (`firstName + " " + lastName`) vào các vị trí đó và nếu thay đổi cài đặt, bạn phải thay đổi từng vị trí như khi bạn muốn có thêm `middleName`. Nhưng khi sử dụng phương thức `getFullName()` thì bạn chỉ cần thay đổi ở một vị trí trong mã nguồn mà thôi.

1.3.4 Đa xạ và ràng buộc động

Đa xạ (polymorphism) có nghĩa là cùng một thông điệp có thể được thể hiện một cách khác nhau qua các lớp đối tượng khác nhau. Ví dụ, trong hệ quản lý thư viện, chèn thêm một cuốn sách khác với chèn thêm một bạn đọc hay một nhân viên vì thông tin các đối tượng này được đưa vào và truyền đi khác nhau và sau đó cũng lưu trữ khác nhau. May mắn là các ngôn ngữ lập trình hiện nay cho phép chúng ta chúng ta xử lý tính đa xạ này thông qua *ràng buộc động* (dynamic binding). Ràng buộc động là kỹ thuật cho phép hoãn định kiểu một đối tượng cho đến thời gian chạy. Nghĩa là một phương thức thực sự được gọi khi chương trình chạy. Ngược lại, trong ràng buộc tĩnh kiểu của đối tượng được xác định tại thời điểm biên dịch và do đó người phát triển phải tự chọn phương thức nào được gọi thay vì để hệ thống tự thực hiện.

Ví dụ trong ngôn ngữ lập trình truyền thống như C, chúng ta phải viết một logic quyết định bằng cách sử dụng các toán tử `if` hay `case` để xác định đối tượng nào cần chèn thêm và phải gọi tên hàm tương ứng (thêm sách, thêm bạn đọc hay thêm nhân viên). Tuy nhiên, trong lập trình hướng đối tượng, chúng ta có thể thiết kế chương trình để cho hệ thống tự lựa chọn hàm thực thi tương ứng vào thời gian chạy.

1.3.5 Quan hệ giữa các lớp

1.3.5.1 Quan hệ liên kết, kết hợp và hợp thành

Không có một đối tượng nào có thể tồn tại và hoạt động riêng lẻ. Tất cả các đối tượng đều được liên kết với các đối tượng khác một cách trực tiếp hoặc gián tiếp, mạnh hoặc

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

yếu. Các liên kết giúp ta tìm ra nhiều thông tin liên quan và hành vi phụ thuộc. Ví dụ, nếu ta đang xử lý một đối tượng khách hàng *Customer* có tên Ngọc và ta muốn gửi cho Lan một bức thư thì ta cần phải biết Ngọc đang sống ở số nhà 168 Nguyễn Trãi, Hà nội. Như vậy, ta muốn có thông tin về địa chỉ được lưu trữ trong đối tượng *Address*, vì vậy cần có kết nối giữa *Customer* và *Address* để biết thư được gửi đến đâu.

Trong mô hình hóa đối tượng với UML, quan hệ giữa các đối tượng được thể hiện theo ba cách chính sau đây: liên kết, kết hợp và hợp thành. Không phải dễ dàng để xác định chính xác sự khác biệt giữa ba quan hệ này, sau đây là một số gợi ý:

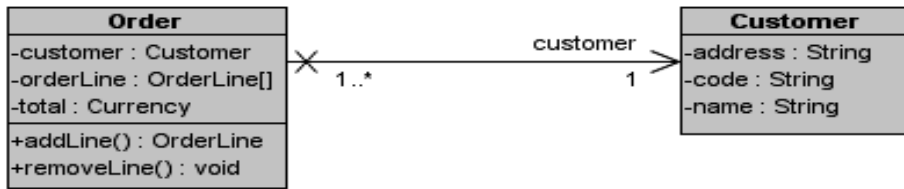
- *Liên kết* (association) là một dạng quan hệ trong đó các đối tượng là một phần của một nhóm, hoặc một họ các đối tượng nhưng chúng không hoàn toàn phụ thuộc vào đối tượng khác. Ví dụ, xét các đối tượng xe hơi, người lái xe và hai khách đi xe. Khi người lái xe và hai khách đi xe ở trong xe, họ được liên kết với nhau vì họ cùng đi về một hướng, họ cùng chiếm một không gian...nhưng liên kết sẽ mất khi xe trả một vị khách nào đó ở nơi yêu cầu, vị khách đó sẽ không còn liên kết với các đối tượng khác nữa.
- *Kết hợp* (aggregation): nghĩa là đặt các đối tượng có liên kết với nhau để tạo thành một đối tượng lớn hơn. Ví dụ, máy tính được tạo bởi các bộ phận như màn hình, ổ cứng, bàn phím... Kết hợp thường có dạng phân cấp *bộ phận-toàn thể* (part-whole). Kết hợp chỉ chỉ sự phụ thuộc giữa bộ phận và toàn thể. Ví dụ, màn hình vẫn là màn hình nếu lấy nó ra khỏi máy tính, nhưng máy tính sẽ mất tác dụng nếu thiếu màn hình.
- *Hợp thành* (Composition). Là dạng mạnh hơn của kết hợp trong đó bộ phận phụ thuộc vào toàn thể một cách duy nhất và đối tượng toàn thể có trách nhiệm tạo lập và hủy bỏ đối tượng bộ phận. Như vậy, khi đối tượng toàn thể bị hủy thì đối tượng bộ phận cũng hủy theo.

Như đã trình bày, không có một ranh giới rõ ràng để phân biệt giữa các quan hệ này đặc biệt quan hệ kết hợp và hợp thành. Nhưng không phải việc phân biệt này lúc nào cũng có thể giải quyết được vấn đề mà cần phải suy nghĩ thường xuyên và cần có kinh nghiệm. Việc lựa chọn này ảnh hưởng rất nhiều đến cách ta thiết kế phần mềm sau này.

Ví dụ: Vấn đề quản lý hóa đơn bán hàng liên quan đến hóa đơn *Order*, các mặt hàng khách hàng đặt *OrderLine*, Danh sách hóa đơn *OrderList* và khách hàng *Customer*. Có nhiều cách cài đặt cho các quan hệ này, ở đây chúng ta sẽ trình bày một cách cài đặt để minh họa rõ hơn sự khác biệt của quan hệ này.

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

Liên kết: Quan hệ liên kết thường được miêu tả giống như quan hệ tham chiếu (reference), trong đó một đối tượng nắm giữ một tham chiếu đến đối tượng khác.



Hình 1.1: Quan hệ liên kết

Có thể cài đặt với Java như sau:

```
package relation;
public class Customer {
    private String address;
    private String code;
    private String name;
}
package relation;
public class Order {
    private Customer customer;
    private OrderLine[] orderLine;
    private Currency total;
    public OrderLine addLine() {
        throw new UnsupportedOperationException();
    }
    public void removeLine() {
        throw new UnsupportedOperationException();
    }
}
```

Đây là quan hệ dễ cài đặt nhất, trong UML nó được biểu diễn bởi một đường thẳng nối giữa hai lớp. Chiều mũi tên nói lên rằng ta gọi đối tượng Customer từ đối tượng Order nhưng không gọi Order từ Customer.

Kết hợp: Quan hệ giữa lớp OrderList và lớp Order thuộc kiểu kết hợp. Nghĩa là danh sách OrderList bao gồm nhiều Order nhưng các Order có đời sống riêng của nó và không cần phải là một bộ phận của danh sách OrderList cụ thể.

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG



Hình 1.2: Quan hệ kết hợp

Cài đặt:

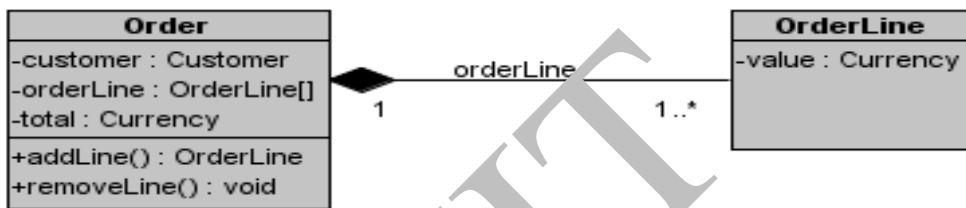
```
package relation;
public class Order {
    private relation.Customer customer;
    private OrderLine[] orderLine;
    private Currency total;
    public OrderLine addLine() {
        throw new UnsupportedOperationException();
    }
    public void removeLine() {
        throw new UnsupportedOperationException();
    }
}

package relation;
import java.util.Vector;
import aggregation.Order;
public class OrderList {
    Vector<Order> order = new Vector<Order>();
    public void add() {
        throw new UnsupportedOperationException();
    }
    public int getCount() {
        throw new UnsupportedOperationException();
    }
    public OrderIterator getIterator() {
        throw new UnsupportedOperationException();
    }
    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

Quan hệ kết hợp được biểu diễn trong UML bởi một đường thẳng có hình quả trám rỗng ở một đầu. Điều này xác định không có quan hệ sở hữu trong quan hệ này và các thể hiện của lớp được kết hợp sẽ được quản lý bên ngoài lớp kết hợp. Gian nhỏ chứa Customer chỉ ra một giới hạn, trong hàm khởi tạo của lớp OrderList có một tham số là Customer để giới hạn số lượng Order tương ứng với Customer đó trong Danh sách OrderList.

Hợp thành

Quan hệ giữa Order và OrderLine thuộc kiểu hợp thành. Các OrderLine của một Order đều thuộc về Order và không có ý nghĩa bên ngoài Order đó. Order có trách nhiệm hoàn toàn trong việc tạo, quản lý và xóa bất kỳ OrderLine nào trong Order đó. Trong UML, quan hệ này được biểu diễn bởi đường thẳng với một đầu có hình quả trám màu đen.



Hình 1.1: Quan hệ hợp thành

Cài đặt:

```

package relation;
public class OrderLine {
    private Currency value;
    aggregation.Order orderLine;
}

package relation;
public class Order {
    private Customer customer;
    private OrderLine[] orderLine;
    private Currency total;
    aggregation.OrderList unnamedOrderList_;
    public OrderLine addLine() {
        throw new UnsupportedOperationException();
    }
    public void removeLine() {
        throw new UnsupportedOperationException();
    }
}
    
```

}

1.3.5.2 Kế thừa

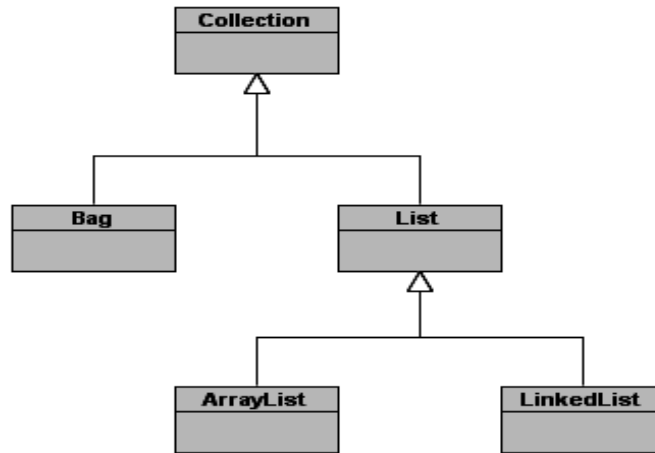
Kế thừa (inheritance) là khái niệm đã được đề xuất và sử dụng rộng rãi trong xây dựng mô hình dữ liệu của những năm 70, 80 thế kỷ trước. Nó cho phép ta thực hiện cách tư duy *đặc biệt hóa* (specialization) khi xây dựng một lớp mới bằng cách lấy lại một số đặc điểm, thuộc tính từ lớp cha và sau đó thêm vào một số đặc trưng riêng của nó. Kế thừa cũng cho phép ta thực hiện cách tư duy *tổng quát hóa* (generalization) bằng cách nhóm một số lớp thành một lớp tổng quát hơn để có thể đưa ra các đối tượng rộng hơn về thế giới mà ta đang sống. Để hiểu một cách đầy đủ hơn khái niệm kế thừa, chúng ta hãy xem xét một ví dụ thiết kế bộ sưu tập để lưu trữ các đối tượng cho sử dụng về sau này.

Thiết kế kiến trúc phân cấp lớp

Sau khi phân tích, chúng ta thấy có bốn kiểu bộ sưu tập như sau:

- *Danh sách (List)*: lưu giữ các đối tượng theo thứ tự mà nó được đưa vào.
- *Túi (Bag)*: lưu đối tượng nhưng không theo thứ tự.
- *Danh sách liên kết (LinkedList)*: lưu các đối tượng theo thứ tự bằng cách cài đặt một chuỗi các đối tượng, mỗi đối tượng chỉ tới một đối tượng khác trong chuỗi. Danh sách liên kết cho phép cập nhật dễ dàng, nhưng truy cập chậm vì ta phải duyệt toàn bộ danh sách.
- *Danh sách mảng (ArrayList)*: lưu các đối tượng theo thứ tự sử dụng như là một mảng, nghĩa là một chuỗi các ô nhớ liên tiếp. Mảng cho phép truy cập nhanh nhưng cập nhật chậm vì ta có thể phải dịch các phần tử hoặc tạo một mảng mới mỗi khi cập nhật.

Làm thế nào thiết kế kiến trúc phân cấp lớp theo kiểu kế thừa? Điểm mấu chốt là cần phải xem xét sự tương đồng giữa các khái niệm với nhau. Rõ ràng, chúng đều là các bộ sưu tập, vì vậy lớp Collection sẽ đưa lên đầu. Trong bốn bộ sưu tập trên, chỉ có Bag là không lưu các đối tượng theo thứ tự, còn lại đều lưu đối tượng theo thứ tự. Do đó, ta đặt Bag trực tiếp ngay bên dưới Collection trong một nhánh riêng. Ta thấy, List không có ràng buộc gì về cài đặt bên trong, trong khi LinkedList và ArrayList thì có. Vì vậy, List sẽ là lớp cha của LinkedList và ArrayList. Như vậy, ta có cây phân cấp sau đây (Hình 1.4):



Hình 1.4: Thiết kế kiến trúc phân cấp

Trong thực tế, ta lại thường hay làm ngược lại. Trước hết, ta mô tả các lớp ở mức lá (Bag, ArrayList và LinkedList) và sau đó, tìm các khái niệm tổng quát hơn. Trong khi phát triển mô hình kế thừa, ta tìm các *thông điệp* để có thể chia sẻ, đưa chúng vào mô hình kế thừa càng ở các lớp trên càng tốt. Ta sẽ tìm các thông điệp trước khi tìm các thành phần lớp khác vì các thông điệp biểu diễn giao tiếp giữa các đối tượng với thế giới bên ngoài. Xét các thông điệp trong mô hình phân cấp Collection:

contains(:Object): boolean tìm các đối tượng trong bộ sưu tập và trả về true nếu bộ sưu tập chứa tham số, ngược lại trả về false.

elementAt(:int): Object trả về đối tượng ở vị trí được xác định bởi tham số truyền vào.

numberOfElements(): int trả về số nguyên là số đối tượng trong bộ sưu tập.

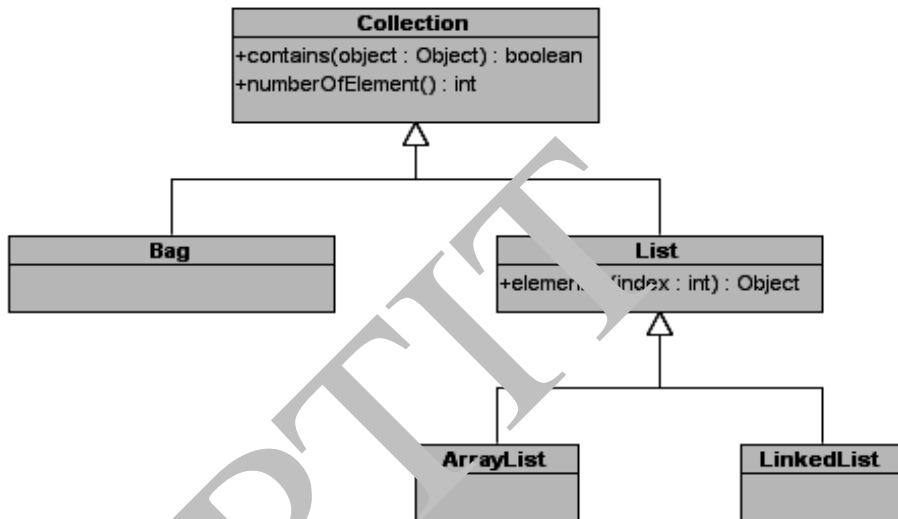
Đặt các thông điệp này vào lớp nào? *contains()* có thể dùng đối với mọi bộ sưu tập, vì vậy đặt nó trong Collection. *elementAt(: int)* lấy đối tượng ở vị trí xác định nên phải đặt trong List, để tránh sự lặp lại nếu để trong ArrayList và LinkedList. *numberOfElement()* có thể dùng với mọi bộ sưu tập nên để nó trong Collection. Ta có cây phân cấp trình bày trong Hình 1.5.

Cài đặt các phương thức trong phân cấp lớp

Vì thuật toán tìm kiếm sẽ xử lý khác nhau đối với bộ sưu tập theo thứ tự và không theo thứ tự, nên *contains()* không thể cài đặt trong Collection. Ở Bag và List ta sẽ cài đặt hàm *contains()* khác nhau.

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

```
//Cài đặt hàm contains trong lớp List
boolean contains(Object obj) {
    for (int i= 0; i< numberOfElements(); ++i) {
        if (elementAt(i) == 0) {
            return true;
        }
    }
    return false;
}
```

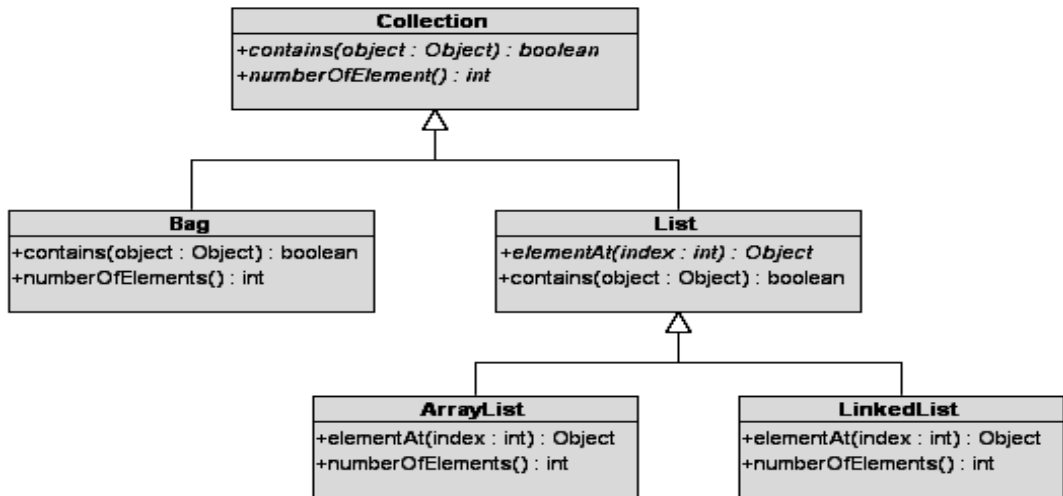


Hình 1.1. Đưa các thông điệp vào các lớp

Phương thức *elementAt* được cài đặt khác nhau trong hai lớp *ArrayList* và *LinkedList*. Vì vậy, ta phải có hai phương thức *elementAt* riêng, một cho lớp *ArrayList* – truy cập các phần tử một cách trực tiếp, một cho lớp *LinkedList* – duyệt toàn bộ danh sách. Cài đặt phương thức *numberOfElements()* phụ thuộc vào việc ta lưu số phần tử trong một trường hay tính số phần tử khi cần.

- *Lưu số phần tử trong một trường*: trường này sẽ tăng khi thêm phần tử và giảm khi xóa phần tử. Cách này cho phép ghi số phần tử một cách nhanh chóng tùy thuộc vào bộ nhớ nhưng chậm trong việc thêm và xóa đối tượng.
- *Tính toán số phần tử khi cần*: đối với *LinkedList* thì chậm vì phải duyệt toàn bộ số phần tử. Đối với *ArrayList* và *Bag*, các đối tượng bên trong có thể lưu trữ số phần tử, do đó sẽ nhanh hơn. Cách này sẽ không tốn bộ nhớ và không bị chậm trong việc thêm và xóa đối tượng.

Ta có thiết kế phân cấp lớp như sau:



Hình 1.6: Đưa các thông điệp vào các lớp

Các phương thức in nghiêng là các phương thức ảo, còn lại là các phương thức thực. Phương thức ảo chỉ có tên mà không có số dòng mã cụ thể, còn phương thức thực thì ngược lại.

Các lớp ảo

Lớp ảo là lớp có ít nhất một phương thức ảo – nó có thể nằm trong lớp đó hoặc được kế thừa từ lớp cha. Khi thiết kế phân cấp lớp, ta nên hình dung trong đầu rằng lớp cha cao nhất là ảo.

Định nghĩa lại các phương thức

Hướng đối tượng cho phép ta định nghĩa lại các phần tử dựa trên kế thừa. Ở dạng đơn giản nhất, định nghĩa lại cho phép lớp con thay đổi việc cài đặt phương thức được kế thừa. Tên phương thức vẫn như cũ nhưng các dòng code trong thân sẽ được thay thế hoặc tạo chuyển thông điệp từ private sang public hoặc đổi tên hoặc kiểu của một thuộc tính...

Sau đây, ta sẽ tập trung bàn về định nghĩa lại nội dung của phương thức, vì đó là lý do quan trọng nhất cho việc định nghĩa lại. Có ba lý do chính giải thích tại sao ta phải định nghĩa lại:

- Phương thức được kế thừa là ảo và ta muốn biến nó thành hiện thực bằng cách thêm vào một số dòng mã. Ví dụ, contains là ảo trong Collection nhưng cần là thực trong Bag và List.
- Phương thức cần phải thực hiện thêm một số công việc khi nằm ở lớp con.

- Ta có thể cung cấp một cài đặt tốt hơn cho lớp con. Ví dụ, nếu thêm một chỉ số vào lớp LinkedList, ta có thể định nghĩa lại contains để làm việc nhanh hơn thuật toán tuần tự được dùng với List.

Khi ta thêm công việc, phải chắc chắn rằng định nghĩa lớp cha vẫn làm mọi thứ bình thường – để tăng việc chia sẻ mã nguồn và đơn giản hóa việc bảo trì (ví dụ, nếu sửa đổi định nghĩa của lớp cha, lớp con sẽ tự động có hành vi mới). Mỗi ngôn ngữ hướng đối tượng cho phép phương thức được định nghĩa lại có thể gọi phương thức trong lớp cha.

```
//Ví dụ trong Java:  
void initialize() {  
    //invoke the inherited initialize method  
    super.initialize();  
    ...  
}
```

Đa kế thừa Mỗi lớp con có nhiều lớp cha. Java có một dạng đa kế thừa đối với interface và lớp abstract (không cài đặt phương thức).

1.4 SỬ DỤNG LẠI

Sử dụng lại (reuse) là khái niệm đã được bàn cãi rất nhiều trong công nghiệp phần mềm. Nhiều nghiên cứu và thực tế phát triển phần mềm đã chỉ ra rằng sử dụng lại dẫn đến phát triển nhanh hơn, hiệu quả hơn và đáng tin cậy hơn vì mã đã được kiểm thử nhiều lần. Hơn nữa, việc bảo trì cũng trở nên dễ dàng hơn. Chúng ta có thể liệt kê ra đây một số cách để sử dụng lại mã nguồn:

- *Sử dụng lại các chức năng trong một hệ thống*: Dạng đơn giản nhất là dùng lại mã nguồn (được sử dụng trong phát triển các hệ thống theo cách truyền thống) liên quan đến việc viết các hàm tiện ích được gọi từ nhiều nơi. Ví dụ, một số môđun trong hệ thống cần sử dụng chức năng tìm kiếm thông qua một danh sách tên khách hàng, do đó có thể viết một hàm tìm kiếm chung để có thể gọi từ các tình huống khác nhau.
- *Sử dụng lại các phương thức trong một đối tượng*: Các phương thức được đóng gói trong một đối tượng có thể được gọi từ các phương thức khác. Ví dụ, trong Java phương thức không public có thể sử dụng trong lớp nào thuộc cùng gói với nó. Bạn nên nghĩ đến việc sử dụng lại các phương thức trong một đối tượng bất cứ khi nào cần.
- *Sử dụng lại các lớp trong một hệ thống*: Nhiều lớp đã định nghĩa có thể được dùng trong các phần khác nhau của hệ thống. Ví dụ, nếu bạn xây dựng lớp khách

CHƯƠNG 1. CƠ SỞ CỦA PHÁT TRIỂN PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

hàng Customer trong hệ thống marketing, bạn cũng muốn các đối tượng Customer tương tự xuất hiện trong nhiều phần khác nhau của mã nguồn hệ thống. Kiểu dùng lại này là cơ sở của cách tiếp cận hướng đối tượng.

- *Sử dụng lại các chức năng trong một số hệ thống:* Các chức năng chung có thể được sử dụng lại (trong phát triển hệ thống kiểu truyền thống cũng như hướng đối tượng) trong các hệ thống khác mà bạn và các đồng nghiệp tạo ra. Với một chức năng để đồng nghiệp có thể được dùng lại, ta phải giải thích cho họ hiểu nó. Giải thích có thể đặt vào nơi chứa tài nguyên dùng lại như một cơ sở dữ liệu các chức năng mà các lập trình viên có thể xem xét khi viết mã nguồn mới.
- *Dùng lại các lớp trong một số hệ thống:* Chúng ta cũng có thể sử dụng lại toàn bộ lớp (thuộc tính và phương thức) hơn chỉ là một phương thức. Ví dụ, ta có thể viết một lớp nhân viên *Employee* với một số thuộc tính cùng với các phương thức để sử dụng cho một số dự án của cùng một công ty.
- *Dùng lại các lớp trong tất cả các hệ thống:* Thành phần phần mềm cũng tương tự như các thành phần phần cứng. Nó có thể tạo ra và sử dụng lại toàn bộ cho nhiều dự án phần mềm. Các thành phần trong phần mềm được thiết kế để có thể dùng lại trong nhiều ngữ cảnh. Nó được đóng gói chặt chẽ (bên yêu cầu không biết công việc bên trong là gì) dùng với tiêu chuẩn của interface và được cung cấp sẵn từ bên thứ ba, thường là phải trả chi phí. Ví dụ, trong Java J2EE, các kiểu *JavaBeans* có thể sử dụng lại cho nhiều hệ phần mềm khác nhau.
- *Các thư viện hàm:* Các hàm liên quan, có chất lượng tốt có thể được nhóm lại thành một thư viện để chi dùng sẵn sàng để sử dụng lại. Ví dụ trong ngôn ngữ C, thư viện *stdio*, bắt nguồn từ các hệ điều hành UNIX, đã cung cấp khả năng I/O cho các lập trình viên C. Các thư viện hàm được dùng cả trong việc phát triển phần mềm truyền thống cũng như phát triển phần mềm hướng đối tượng. Các thư viện được thiết kế tốt đôi khi được chuẩn hóa bởi các tổ chức như ISO hay ANSI. Các thư viện hàm có thể xuất phát từ bên trong công ty, sử dụng miễn phí hoặc bán để kiếm lời.
- *Các thư viện lớp:* Là sự phát triển của các thư viện chức năng, thường là các lớp hoàn chỉnh chứ không đơn thuần là các hàm. Viết một thư viện lớp đòi hỏi có nhiều kinh nghiệm. Ví dụ thư viện J2EE, cung cấp mã nguồn cho tất cả các kiểu dùng lại được liệt kê ở trên.
- *Mẫu thiết kế (design pattern):* Mẫu thiết kế là một sự mô tả cách tạo ra các phần của hệ thống hướng đối tượng một cách hợp lý và hiệu quả. Các mẫu cũng được áp dụng trong các lĩnh vực khác như kiến trúc hệ thống. Mỗi mẫu là một miêu tả

ngắn, chi tiết, cho biết khi nào dùng nó và mã nguồn minh họa. Thiết kế các mẫu đòi hỏi phải có nhiều kinh nghiệm, nhưng không bằng việc tạo ra thư viện lớp.

- *Khuôn dạng (Framework)*: Là một cấu trúc đã có sẵn để bạn gắn mã nguồn của mình vào. Trong hướng đối tượng, một framework bao gồm một số lớp đã được viết trước, cùng với tài liệu miêu tả hướng dẫn lập trình viên các quy tắc phải tuân theo. Ví dụ, EJB framework – bao gồm thư viện J2EE và tài liệu đặc tả, dài hàng trăm trang – chỉ ra cách để lập trình viên viết được các thành phần có khả năng dùng lại, và cách để các bên thứ ba cài đặt máy chủ ứng dụng Java. Phần lớn các framework được thiết kế bởi các chuyên gia cao cấp (guru).

1.5 KẾT LUẬN

Chương này đã trình bày một số vấn đề cơ bản nhất trong phát triển phần mềm hướng đối tượng. Nội dung tập trung vào xem xét những kiểu phần mềm; những đặc trưng cơ bản của các hệ hướng đối tượng như đối tượng, lớp... đến những vấn đề phức tạp như quan hệ giữa các lớp. Những khái niệm này đã được trình bày ngắn gọn với những ví dụ minh họa để bạn đọc dễ hình dung. Thêm vào đó vấn đề sử dụng lại cũng đã được đề cập đến để bạn đọc hiểu được yếu tố quan trọng của công nghiệp phần mềm ngày nay là sử dụng lại.

Tuy nhiên, phân tích và thiết kế theo hướng đối tượng bao gồm nhiều vấn đề cần sự nghiên cứu lâu dài cũng như trải nghiệm thực tế qua nhiều dự án phần mềm mới có thể trở thành chuyên gia trong lĩnh vực này. Các chương tiếp theo sẽ giúp bạn đọc hiểu rõ hơn những vấn đề này.

BÀI TẬP

1. Tìm hiểu và khảo sát các hệ thương mại điện tử, hệ quản lý thư viện, hệ quản lý học tập theo tín chỉ sau đó đề xuất các chức năng của hệ thống này. Liệt kê các tác nhân sử dụng hệ thống.
2. Hãy chọn ra các lớp từ các hệ thống trong câu 1 và chỉ ra các quan hệ giữa chúng.
3. Hãy thêm các thuộc tính và các phương thức vào các lớp chọn được. Giải thích lý do chọn lựa các mức độ truy nhập các thuộc tính trên và lý do gán phương thức vào các lớp.
4. Sử dụng khái niệm interface và abstract trong Java để cài đặt các lớp với phương thức chèn thêm sinh viên, chèn thêm môn học, chèn thêm đăng ký học.
5. Tương tự như các Câu 1 cho hệ quản lý khác.
6. Sinh viên tham khảo thêm các tài liệu để viết một số ví dụ một mẫu thiết kế.

CHƯƠNG 2

MÔ HÌNH HÓA HỆ PHẦN MỀM HƯỚNG ĐỐI TƯỢNG

Chương này trước hết giới thiệu ngôn ngữ mô hình hoá thống nhất và sau đó trình bày các phương pháp luận phát triển phần mềm hướng đối tượng cùng một tiến trình phát triển phần mềm đơn giản dành cho tài liệu này. Nội dung cụ thể bao gồm:

- Giới thiệu UML và các biểu đồ trong UML
- Các phương pháp luận phát triển phần mềm hướng đối tượng
- Giới thiệu công cụ phát triển phần mềm

2.1 GIỚI THIỆU VỀ UML

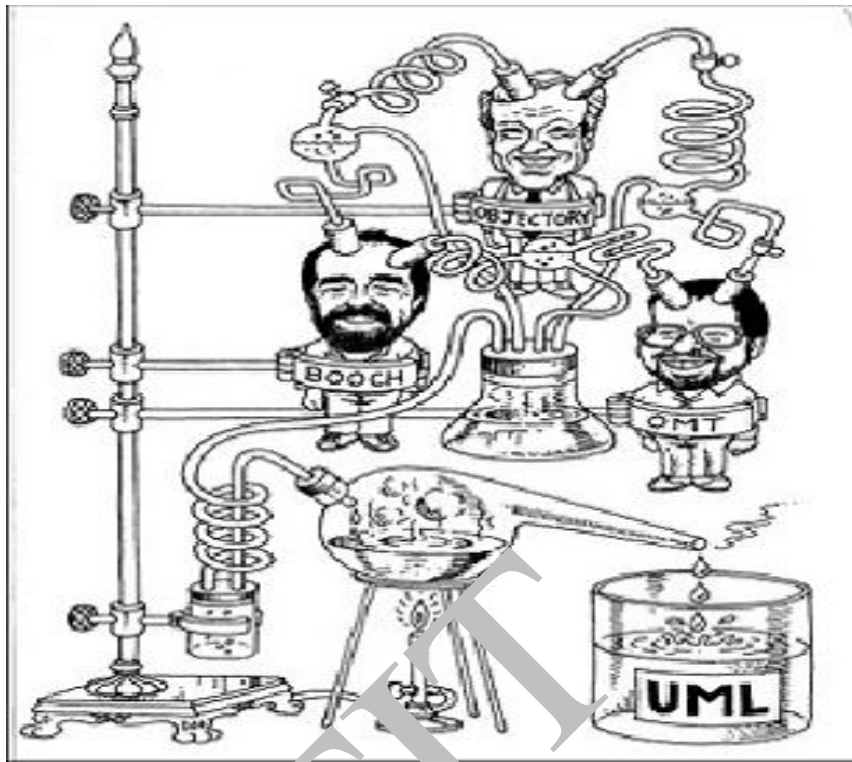
2.1.1 Lịch sử phát triển của UML

Các ngôn ngữ lập trình hướng đối tượng ra đời khá sớm, ví dụ như Simula-67 (năm 1967), Smalltalk (đầu những năm 1980), C++, CLOS (giữa những năm 1980)... Tuy nhiên, mãi cho đến năm 1995, những nhóm phát triển phần mềm mới có những phương pháp luận và ngôn ngữ mô hình hoá có hiệu quả khác nhau, như Booch của Grady Booch, OMT của James Rumbaugh, OOSE của Ivar Jacobson, hay OOA & OOD của Coad và Yordon... Việc áp dụng rộng rãi phương pháp hướng đối tượng đã đặt ra nhu cầu phải xây dựng một ngôn ngữ mô hình hoá thống nhất như một chuẩn chung cho những người phát triển phần mềm hướng đối tượng trên khắp thế giới. Nỗ lực thống nhất đầu tiên bắt đầu khi Rumbaugh gia nhập nhóm nghiên cứu của Booch tại tập đoàn Rational năm 1994 và sau đó Jacobson cũng gia nhập nhóm này vào năm 1995.

James Rumbaugh, Grady Booch và Ivar Jacobson đã cùng cố gắng xây dựng một Ngôn Ngữ Mô Hình Hoá Thống Nhất có tên là UML (Unified Modeling Language) (Hình 2.1). Phiên bản UML đầu tiên được đưa ra năm 1997 và sau đó được chuẩn hoá để trở thành phiên bản 1.0. UML đã không ngừng phát triển (chi tiết tham khảo <http://www.omg.org/spec/UML/>) và phiên bản 2.0 đã được sử dụng rộng rãi trong công nghiệp phần mềm hiện nay với nhiều công cụ hỗ trợ.

2.1.2 UML – Ngôn ngữ mô hình hoá hướng đối tượng

UML là ngôn ngữ mô hình hoá được xây dựng để đặc tả, phát triển và viết tài liệu cho các hệ phần mềm hướng đối tượng. UML bao gồm một tập các khái niệm, các ký hiệu, các biểu đồ và hướng dẫn sử dụng.



Hình 2.1: Sự ra đời của UML

Mục đích của ngôn ngữ UML là: (i) Mô hình hoá các hệ thống bằng cách sử dụng các khái niệm hướng đối tượng; (ii) Thiết lập sự liên hệ từ nhận thức của con người đến các sự kiện cần mô hình hoá; (iii) Giải quyết vấn đề về mức độ thừa kế trong các hệ thống phức tạp với nhiều ràng buộc khác nhau; (iv) Tạo một ngôn ngữ mô hình hoá có thể sử dụng được bởi người và máy. UML hỗ trợ phân rã hệ hướng đối tượng dựa trên cấu trúc tĩnh và hành vi động của hệ thống.

- Các cấu trúc tĩnh (static structure) xác định các kiểu đối tượng quan trọng của hệ thống và mối quan hệ giữa các đối tượng đó nhằm đến cài đặt sau này.
- Các hành vi động (dynamic behavior) xác định các hành động của các đối tượng theo thời gian và tương tác giữa các đối tượng.

2.1.3 Các khái niệm cơ bản trong UML

Khái niệm mô hình

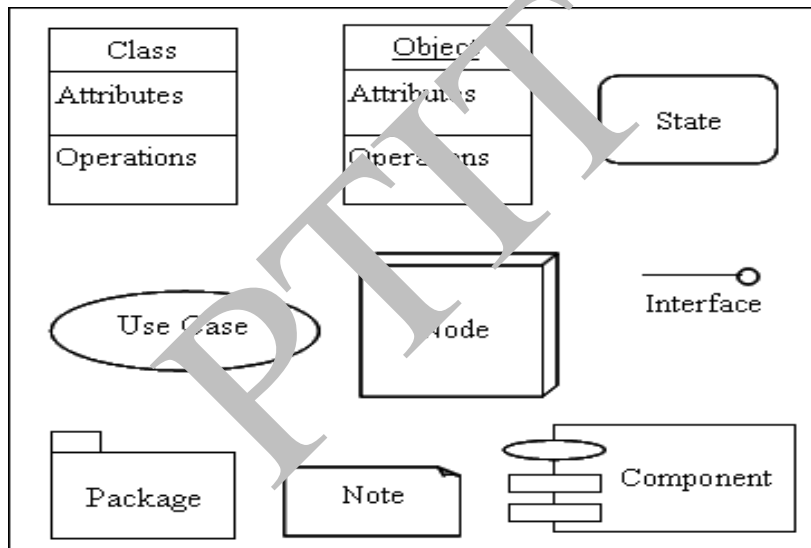
Mô hình (model) là một biểu diễn của sự vật, đối tượng hay một tập các sự vật trong một lĩnh vực ứng dụng nào đó theo một quan điểm nhất định. Mục đích của mô hình là nhằm nắm bắt các khía cạnh quan trọng của sự vật mà mình quan tâm và biểu diễn theo một tập

ký hiệu hoặc quy tắc nào đó. Các mô hình thường được xây dựng sao cho có thể vẽ được thành các biểu đồ dựa trên tập ký hiệu và quy tắc đã cho.

Ví dụ, khi xây dựng Hệ quản lý bán hàng thì ta chỉ cần quan tâm đến các thuộc tính như họ tên, địa chỉ, phone, email...của đối tượng khách hàng. Trong khi xây dựng hệ Quản lý Học tập theo tín chỉ ngoài các thông tin liên quan đến đối tượng sinh viên như họ tên, địa chỉ, email, phone...ta còn phải quan tâm đến các thuộc tính như điểm, lớp học, môn học, khoa mà sinh viên đăng ký.

Các phần tử mô hình và các quan hệ

Một số ký hiệu để mô hình hóa hướng đối tượng thường gặp trong UML được biểu diễn trong Hình 2.2. Đi kèm với các phần tử mô hình này là các *quan hệ*. Các quan hệ này có thể xuất hiện trong bất cứ mô hình nào của UML dưới các dạng khác nhau như quan hệ giữa các ca sử dụng, quan hệ trong biểu đồ lớp...



Hình 2.2: Một số phần tử mô hình thường gặp trong UML

